

Git & GitHub: A Teaching Guide

Git vs GitHub

- **Git** is a version control tool that runs on your computer. It records snapshots (commits) of your project so you can go back in time, compare changes and work on features safely.
- **GitHub** is a website that hosts Git repositories online, so people can back up code, share it and collaborate.

Git works without GitHub. GitHub is just one place to store Git repositories.

The three areas of Git (key idea to teach):

1. **Working directory:** files as you see and edit them.
2. **Staging area:** files you have marked to go into the next commit.
3. **Repository:** the permanent history of commits.

Flow: edit -> `git add` -> `git commit`.

1. Local Version Control Commands

Command	What it does
<code>git --version</code>	Checks whether Git is installed.
<code>git config --global user.name "Your Name"</code>	Sets the name attached to your commits.
<code>git config --global user.email "you@example.com"</code>	Sets the email attached to your commits.
<code>git init</code>	Turns the current folder into a Git repository (creates a hidden <code>.git</code> folder).
<code>git status</code>	Shows which files are modified, staged or untracked. Use it constantly.
<code>git add file.txt</code>	Stages one file for the next commit.
<code>git add .</code>	Stages all changes in the current folder.
<code>git commit -m "message"</code>	Saves the staged changes as a snapshot with a message.
<code>git commit -am "message"</code>	Stages all already-tracked files and commits in one step (does not include new files).
<code>git log</code>	Shows the commit history.
<code>git log --oneline</code>	Shows history in a short one-line-per-commit view.
<code>git diff</code>	Shows unstaged changes line by line.
<code>git diff --staged</code>	Shows what is staged and will go into the next commit.
<code>git show <commit-id></code>	Shows the details and changes of one commit.
<code>git rm file.txt</code>	Deletes a file and stages the deletion.
<code>git mv old.txt new.txt</code>	Renames or moves a file and stages the change.

Typical first workflow:

```
git init
git add .
git commit -m "Initial commit"
```

Tip: `.gitignore`. A file named `.gitignore` lists files Git should never track (passwords, `node_modules/`, `.env`, `__pycache__/`).

Good commit messages are short and describe what changed: "Add login form validation", not "changes" or "stuff".

2. Branching Commands

A **branch** is an independent line of development. The default branch is usually `main`. Branches let you build a feature or fix a bug without touching working code.

Creating and switching

Command	What it does
<code>git branch</code>	Lists local branches (the current one is starred).
<code>git branch feature-login</code>	Creates a new branch (does not switch to it).
<code>git switch feature-login</code>	Switches to an existing branch.
<code>git switch -c feature-login</code>	Creates a branch and switches to it in one step.
<code>git checkout feature-login</code>	Older command for switching (still common).
<code>git checkout -b feature-login</code>	Older way to create and switch.
<code>git branch -d feature-login</code>	Deletes a branch that is already merged.
<code>git branch -D feature-login</code>	Force-deletes a branch, even if not merged.
<code>git branch -m new-name</code>	Renames the current branch.

Merging

To merge `feature-login` into `main`:

```
git switch main
git merge feature-login
```

- **Fast-forward merge:** `main` had no new commits, so Git simply moves the pointer forward.
- **Three-way merge:** both branches have new commits, so Git creates a new merge commit.
- **Merge conflict:** both branches changed the same lines. Git pauses and marks the file:

```
<<<<<< HEAD
your version
=====
other branch's version
>>>>>> feature-login
```

Edit the file to keep what you want, remove the markers, then run `git add file` and `git commit`.

Undoing a merge

Situation	Command
Merge is in progress with conflicts and you want to cancel	<code>git merge --abort</code>
Merge is done but not pushed yet	<code>git reset --hard HEAD~1</code> (removes the merge commit and its changes)
Merge is done and already pushed	<code>git revert -m 1 <merge-commit-id></code> (creates a new commit that undoes it)

`-m 1` tells Git to keep the first parent, which is the branch you merged into (usually `main`).

Reset vs Revert vs Restore: when to use which

These three are the most confusing commands for beginners. Teach them by asking: **"Is it committed? Is it pushed?"**

`git restore: undo changes to files (not history)` Use when you edited a file and want to throw the edit away, or unstage something.

```
git restore file.txt          # discard unstaged changes in the file
git restore --staged file.txt # unstage a file (keep the edits)
git restore --source=<commit-id> file.txt # bring back a file's older version
```

git reset: move the branch backward (rewrites history) Use for commits that exist **only on your computer**.

```
git reset --soft HEAD~1 # undo last commit, keep changes staged
git reset --mixed HEAD~1 # undo last commit, keep changes unstaged (default)
git reset --hard HEAD~1 # undo last commit AND delete the changes (dangerous)
```

git revert: undo a commit by adding a new commit (safe history) Use for commits that are **already pushed/shared**.

```
git revert <commit-id>
```

	restore	reset	revert
Works on	Files	Commits	Commits
Changes history?	No	Yes (rewrites)	No (adds a commit)
Safe on shared branches?	Yes	No	Yes
Typical use	Discard file edits	Undo local commits	Undo pushed commits

Rule of thumb: Not committed -> **restore**. Committed but local only -> **reset**. Already pushed -> **revert**.

Bonus: `git stash` temporarily shelves uncommitted work (`git stash pop` brings it back). Useful when you need to switch branches mid-task.

3. Pushing to GitHub

First, create an empty repository on GitHub (github.com -> New repository). Then connect your local project to it.

Command	What it does
<code>git remote add origin <repo-url></code>	Links your local repo to the GitHub repo. <code>origin</code> is the nickname for that URL.
<code>git remote -v</code>	Shows the remote URLs currently linked.
<code>git branch -M main</code>	Renames the current branch to <code>main</code> .
<code>git push -u origin main</code>	Uploads your commits to GitHub and sets <code>origin/main</code> as the default upstream, so later you can just type <code>git push</code> .
<code>git push</code>	Uploads new commits on the current branch (after upstream is set).
<code>git push origin feature-login</code>	Pushes a specific branch to GitHub.
<code>git push origin --delete feature-login</code>	Deletes a branch on GitHub.
<code>git push --force</code>	Overwrites remote history. Avoid on shared branches.

Full flow for a brand-new project:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/username/my-project.git
git push -u origin main
```

Authentication: GitHub no longer accepts account passwords for Git operations. Use a **Personal Access Token** (over HTTPS) or **SSH keys**.

Everyday routine after setup: `git add .` -> `git commit -m "..."` -> `git push`

4. Pulling and Cloning from GitHub

Command	What it does
<code>git clone <repo-url></code>	Downloads a complete copy of a repo (all files and history) into a new folder. Remote origin is set up automatically.
<code>git clone <repo-url> my-folder</code>	Clones into a folder with a custom name.
<code>git fetch</code>	Downloads new changes from GitHub but does not change your files. Safe way to look first.
<code>git pull</code>	Fetches and merges remote changes into your current branch (fetch + merge).
<code>git pull --rebase</code>	Fetches and replays your commits on top of the remote ones, giving a cleaner linear history.
<code>git pull origin main</code>	Pulls a specific branch from the remote.
<code>git branch -r</code>	Lists remote branches.

Clone vs Pull:

- **Clone** = first time, you don't have the project yet.
- **Pull** = you already have it and want the latest updates.

Fork vs Clone: A **fork** is your own copy of someone else's repo *on GitHub*. You then **clone** your fork to your computer. Forking is how you contribute to projects you don't own.

Good habit: Always `git pull` before you start working and before you push, so you avoid conflicts and rejected pushes.

If push is rejected ("updates were rejected because the remote contains work you do not have"), run `git pull`, resolve any conflicts, then push again.

5. GitHub Branching and Merging (Collaboration)

Why branches matter on GitHub

On a team, nobody works directly on `main`. `main` should always hold stable, working code. Each person creates a branch for their feature or bug fix, works there, and proposes to merge it back. This keeps unfinished work from breaking the project, lets many people work in parallel, and allows review before code goes live.

The Feature Branch Workflow

1. **Pull** the latest `main`.
2. **Create a branch** for your task (name it clearly: `feature/login-page`, `fix/navbar-bug`).
3. **Commit** your work on that branch.
4. **Push** the branch to GitHub.
5. **Open a Pull Request (PR)**.
6. Teammates **review** it and suggest changes.
7. The PR is **merged** into `main`.
8. **Delete** the branch and **pull** the updated `main`.

```
git switch main
git pull
git switch -c feature/login-page
# ...edit files...
git add .
git commit -m "Add login page"
git push -u origin feature/login-page
```

Then continue on the GitHub website to open the Pull Request.

What is a Pull Request?

A **Pull Request** is a request to merge your branch into another branch (usually `main`). It is not a Git command, it is a GitHub feature. A PR shows exactly what changed, provides a place for discussion and review comments, can run automated checks (tests), and records why a change was made. Think of it as saying: *"I've finished this work, please review it and pull it into main."*

Ways to merge a PR on GitHub

Option	What happens	When to use
Create a merge commit	All commits are kept, plus a merge commit.	You want the full history preserved.
Squash and merge	All commits are combined into one commit.	Many small "fix typo" commits; you want a clean history.
Rebase and merge	Commits are replayed on top of <code>main</code> without a merge commit.	You want a straight, linear history.

Merge conflicts on GitHub

If two branches change the same lines, GitHub shows "This branch has conflicts". You can resolve small ones in the web editor, or locally:

```
git switch feature/login-page
git pull origin main          # brings main's changes into your branch
# fix conflict markers in files
git add .
git commit -m "Resolve merge conflicts"
git push
```

The PR then becomes mergeable.

Keeping your branch up to date

While you work, `main` may move ahead. Regularly bring those changes into your branch:

```
git fetch origin
git merge origin/main
```

Cleaning up after a merge

```
git switch main
git pull
git branch -d feature/login-page      # delete local branch
git push origin --delete feature/login-page # delete remote branch (GitHub also offers a button)
```

Protecting `main`

Repository owners can enable **branch protection rules** (Settings -> Branches): require PR reviews, require passing tests, and block direct pushes to `main`.

Rebase (brief intro)

`git rebase main` moves your branch's commits to start from the tip of `main`, producing a linear history. It **rewrites commits**, so never rebase branches that others are using.

Quick Cheat Sheet

Goal	Command
Start a repo	<code>git init</code>
Stage and commit	<code>git add .</code> then <code>git commit -m "msg"</code>

Goal	Command
See status / history	<code>git status, git log --oneline</code>
New branch and switch	<code>git switch -c name</code>
Merge a branch	<code>git switch main</code> then <code>git merge name</code>
Connect to GitHub	<code>git remote add origin <url></code>
Upload commits	<code>git push -u origin main</code>
Download a repo	<code>git clone <url></code>
Get latest changes	<code>git pull</code>
Discard file edits	<code>git restore file</code>
Undo local commit	<code>git reset --soft HEAD~1</code>
Undo pushed commit	<code>git revert <id></code>

Suggested Class Exercises

1. Create a folder, run `git init`, make three commits, and explore them with `git log`.
2. Create a branch, add a file, and merge it into main.
3. Deliberately create a merge conflict (edit the same line in two branches) and resolve it.
4. Push a project to GitHub, then have a partner clone it.
5. Each student forks a practice repo, makes a change on a branch and opens a Pull Request.